

Agentic AI Customer 360

End-Term Solution Report

Atharva Pandey — IIT (BHU), Varanasi

1. Problem and Approach

The objective of the system is to maintain a continuously updated Customer 360 view from heterogeneous customer events and use the evolving state to identify situations where a proactive intervention may be useful.

Instead of forwarding the complete event history to one general-purpose model, the system separates the problem into specialist agents, shared customer state, memory, synthesis, higher-level inference, bounded action selection, refinement, guardrails, and human approval.

1.1 Final System Architecture

Figure 1 shows the implemented end-to-end pipeline. The design uses parallel specialist analysis, structured handoffs, conditional disagreement resolution, sequential action refinement, and a human approval boundary before consequential execution.

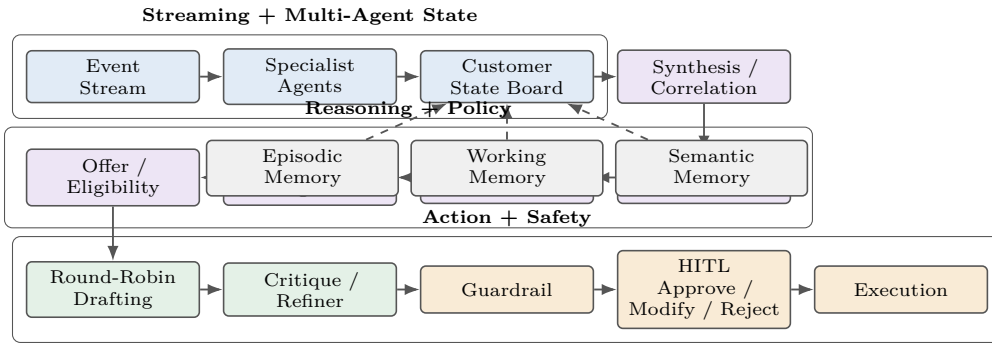


Figure 1: Final Agentic Customer 360 architecture. Solid arrows represent the main execution path; dashed arrows represent conditional debate and memory/state access.

The central architectural idea is to use different coordination patterns for different tasks instead of forcing a single orchestration pattern across the complete system.

2. Streaming Event Processing and Customer State

The system processes multiple event sources, including transaction, support, application-usage, and KYC/compliance events.

Event time and ingestion time are maintained separately. This allows the stream processor to distinguish an event that happened earlier from the time at which that event was received. Late or out-of-order events can therefore be detected and incorporated without silently corrupting the customer state.

Each accepted event is routed to the specialist responsible for its domain. The specialist publishes a structured finding to a shared per-customer state board rather than passing its complete internal context to every other agent.

A finding contains structured information such as:

- finding type,
- confidence,
- supporting evidence,
- source event identifiers,
- event time,
- relevant customer context.

This creates controlled handoffs and prevents context pollution.

2.1 Specialist Agents

The specialist layer contains four main agents:

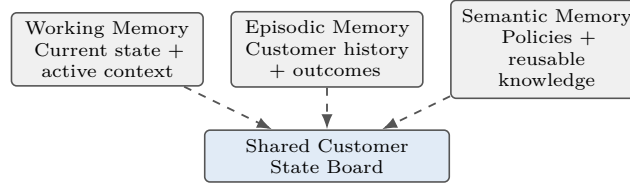
- **Transaction Agent:** financial activity, transfers, payments, spending behaviour, and transaction changes.
- **Support Agent:** support tickets, customer messages, urgency, and support-related signals.
- **Usage Agent:** logins, searches, application usage, and digital engagement.
- **KYC/Compliance Agent:** profile changes and compliance-related information.

The agents have different responsibilities and publish scoped structured findings to the customer state board. This

follows a blackboard-style coordination model in which agents share conclusions rather than unrestricted internal context.

3. Memory Architecture

The implementation separates customer information into three complementary memory layers.



The main purpose of this separation is to avoid repeatedly placing the entire customer history into every prompt. Relevant information can instead be retrieved according to the current task. This follows the context-management motivation studied in memory-oriented agent systems while adapting the idea to a shared per-customer state-board architecture.

The memory design also supports traceability because retrieved memories can be linked to the resulting inference or action.

4. Synthesis, Life-Event Inference, and Debate

The Synthesis/Correlation stage combines the structured findings produced by the specialist agents. This allows multiple weak signals to be interpreted together instead of reacting to a single event in isolation.

The resulting correlated context is then passed to the Life-Event Inference stage. The purpose of this stage is to infer a higher-level customer situation from the combined evidence and available customer context.

The system maintains an explicit distinction between:

raw events $\rightarrow$ specialist evidence $\rightarrow$ correlated signals $\rightarrow$ higher-level inference.

This separation makes the reasoning process easier to inspect and explain.

When the specialist outputs provide materially conflicting interpretations, the system can activate a dedicated Debate and Resolution stage. Each side receives scoped evidence and produces a structured position. A resolution step then produces the state passed to downstream decision making.

The debate stage is conditional rather than mandatory. This avoids unnecessary model calls when the evidence is already consistent while preserving a mechanism for disagreement.

5. Decision, Eligibility, and Action

After the customer state is inferred, the Decision stage selects a bounded intervention from the allowed action vocabulary.

The action is represented together with an action subtype and supporting rationale. This prevents downstream agents from converting a high-level inference into an unrestricted or unsupported action.

The Offer/Eligibility stage then checks whether the proposed intervention is allowed under the current policy and customer context.

This gives the system a separation between:

What is happening? and What may the system do about it?

Policy and eligibility constraints are therefore kept separate from higher-level customer-state reasoning.

6. Round-Robin Drafting and Critique

For customer-facing actions, the system does not directly execute the first generated response.

Instead, several role-specific agents generate candidate drafts. The demonstrated pipeline includes:

- Customer Support Agent,
- Relationship Manager Agent,
- Customer Experience Agent.

The drafts are kept as separate artifacts so that different perspectives can be compared.

The pipeline then enters a Critique/Compliance-Refiner stage. The refiner examines the proposed drafts for policy alignment, risky wording, unsupported claims, tone, and other compliance concerns.

The resulting workflow is:

Generate → Compare → Refine → Validate.

This separation is important because the system does not treat message generation and message approval as the same operation.

7. Guardrails and Human-in-the-Loop

A deterministic guardrail is placed immediately before consequential execution.

The guardrail checks hard-stop conditions and determines whether the proposed action can proceed automatically or must be routed to a human reviewer.

For customer-facing or otherwise consequential actions, the Human-in-the-Loop stage provides an explicit interruption in the execution path.

The reviewer can choose:

- **Approve:** continue toward execution.
- **Modify:** edit the proposed action or message and revalidate it.
- **Reject:** prevent execution.

The HITL decision is persisted in the audit trail together with the proposed action, customer context, evidence, and final outcome.

This means HITL is not merely a status printed in the terminal. It is an actual checkpoint that controls whether execution is allowed to continue.

8. Explainability, Traceability, and Observability

The system records the evidence supporting important inferences and connects that evidence to source event identifiers.

The resulting provenance chain is:

Source Events → Agent Findings → Correlated Signals → Inferred State → Action → Drafts → Critique → HITL → Execution

The implementation also produces timestamped trace records, inferred-event records, HITL audit records, and evaluation artifacts.

This provides two benefits. First, the system can explain why a customer state or action was produced. Second, a reviewer can reconstruct the complete decision path after execution.

Observability is therefore treated as part of the architecture rather than as a debugging-only feature.

9. Streaming Robustness

The replay pipeline preserves event time independently from ingestion time.

When an event arrives late, the system records the late arrival and rebuilds the affected customer state using event-time ordering. Duplicate events are detected using event identity so that repeated deliveries do not silently double-count customer activity.

A representative recorded scenario contained:

Historical events processed	375
Live events processed	116
Total state events	491
Late events detected	1

This demonstrates that the system is designed for an event-stream setting rather than a one-shot batch input.

10. Evaluation

The project includes an evaluation harness that consumes the generated inferred-events artifact and compares system outputs with the practice-scenario expectations.

Evaluation is performed at the checkpoint level and also at the final terminal result.

10.1 Recorded Scenario Results

Scenario	Inferred State	Action	Recorded Result
Scenario 01	Medical-hardship evidence appears in the stream	Support-intervention path is the expected actionable path	Latest checkpoint evaluation was affected by provider availability
Scenario 02	New-child life event	Personalized offer	Final state, action, and subtype matched the recorded expectation
Scenario 03	Churn risk	Proactive retention outreach	Final state and action matched the recorded expectation

The recorded `scenario_02` evaluation reports the final predicted state as `new_child_life_event`, with action `personalized_offer` and subtype `childcare_savings_or_insurance_plan`. The final state, action, and subtype checks are marked as passing.

The recorded `scenario_03` evaluation reports the final predicted state as `churn_risk`, with action `proactive_retention_outreach` and the final state and action checks are marked as passing.

The evaluation artifacts also exposed an engineering limitation: repeated checkpoint execution can be affected by external model-provider availability. This is treated as a deployment and robustness consideration rather than hidden from evaluation.

10.2 Sample Inferred-Events Output

The system writes a machine-readable inferred-events artifact during stream processing. Each record contains the event-time checkpoint, inferred customer state, confidence, action, action subtype, and Human-in-the-Loop status.

Scenario	Checkpoint	Conf.	Band	Inferred State	Action / Subtype	HITL
S01	2026-02-12 14:20	0.40	Low	No significant event	No action / Monitor	Auto-approved
S01	2026-03-22 18:05	0.40	Low	No significant event	No action / Monitor	Auto-approved
S02	2026-03-25 13:10	0.95	High	New-child life event	Personalized offer / Childcare savings or insurance plan	Human-approved
S03	2026-04-03 09:15	0.95	High	Churn risk	Proactive retention outreach / Customer retention review	Auto-approved

The table is a representative view of the generated artifact rather than the complete event log. The complete inferred-events file is retained separately for checkpoint-level evaluation and reproducibility.

11. Research to Implementation

The final system connects the main research directions studied during the project to concrete architectural choices.

Research / Pattern	Implementation Decision
Context management and memory systems	Separate working, episodic, and semantic memory with selective retrieval.
Blackboard / shared-state architecture	Per-customer state board containing structured specialist findings.
Multi-agent orchestration	Independent specialist analysis followed by structured sequential reasoning.
Multi-Agent Debate	Conditional debate and resolution when meaningful disagreement occurs.
Human-in-the-Loop workflows	Approve / Modify / Reject checkpoint before consequential execution.
Multi-agent drafting	Multiple role-specific agents generate alternative customer-facing drafts.

The main architectural contribution is therefore not the use of an LLM in isolation, but the composition of streaming state, specialist agents, shared memory, structured handoffs, conditional coordination, action generation, critique, guardrails, human oversight, and evaluation.

12. Conclusion

The resulting system is an end-to-end Agentic Customer 360 prototype rather than a single customer classifier.

It accepts heterogeneous customer events, maintains a continuously updated per-customer state, routes events to specialized agents, combines their findings through synthesis, performs higher-level life-event reasoning, and converts that reasoning into bounded actions.

For consequential actions, the system continues through multiple drafting perspectives, critique and refinement, deterministic guardrails, and a real Human-in-the-Loop approval checkpoint before execution.

The architecture also addresses late and duplicate events, memory separation, structured agent handoffs, conditional disagreement handling, source-linked explainability, traceability, and machine-readable evaluation output.

The practice scenarios demonstrate that the complete pipeline can be exercised end-to-end, while the evaluation artifacts also identify provider availability and checkpoint timing as practical limitations. This makes the implementation measurable and inspectable rather than presenting only a final demo output.